

ADC-Project 2025/2026, 2º Semester
Individual Evaluation Exercise

Individual Evaluation Exercise: Requirements and Specification

1. Requirements and Specifications

For the required exercise, students will develop a project following the **mandatory functional requirements** described in this document. Beyond the compliance with the mandatory requirements, each student is free to add additional features considered interesting to further demonstrate their learning achievements.

Application to be Developed

The application will include two main components (as described below):

- **CLIENT component**
- **CLOUD-ENABLED SERVER component**

1.1 Cloud-Enabled Server Component (or Cloud Service Component)

The server component **must** be deployed in a cloud environment using:

- Google **App Engine** for application execution
- Google Firestore database in **Datastore** mode for persistent storage

Optionally, the solution **may** also use:

- Google **Cloud Storage** for additional data storage

The server component must support **REST endpoints** to handle remote operations requested by the client-side component. The specification of the required REST operations is provided in Section 2.

1.2 Client-Side Component (or Client-Side Tester Component)

The client-side component will be used primarily for testing and demonstrating the REST operations supported by the server component.

1.2.1 Testing tools

To test REST (POST-based) operations, students may use different strategies (as introduced in class):

- Use of Postman
- Optionally, use of command-line tools (such as curl), which can be installed on Linux or macOS, or used in Windows via WSL. The curl command-line tool is also available for Windows, as it has been built in since the April 2018 update (Windows 10) and is included by default in Windows 11.

Example:

Using curl to Test a Remote POST REST Operation in a ref. URL:

```
$ curl -X POST -d '{ id: "9", "name": "hj"}' -H 'Content-Type: application/json' http://xxxxx:port/rest/foos/new
```

In this example, **id** and **name** are sent as input parameters in the request body as JSON properties, similarly to what we have seen with Postman. See other useful links if you want to use curl in command-line tool

- <https://curl.se/download.html>
- Modern Windows 10/11 also includes a native curl.exe in C:\Windows\System32, which can be accessed directly from Command Line
- See examples for Windows in the Annex.

1.2.2 Automatic Tests

To automate testing of multiple POST REST endpoints (instead of invoking a single request each time), you can:

- Use **Collections in Postman**
- Create scripts using curl:
 - Batch (.bat) scripts for Windows
 - Shell scripts for Linux/macOS
- See other additional examples in the **Annex** for testing if you want to implement automatic test scripts.

2. REST operations that must be supported

2.1 Rest Operations

All operations to be implemented must be supported using **POST requests**. The operation is part of the endpoint URL while the structure of the data sent with the request depends on whether a token is required to make proof of an authenticated access. Operations will have the following generic structure:

Operations **not requiring JSON Tokens** to be executed:

- **operation**: operation identifier (<opid>)
- **input**: JSON input data (arguments for the POST operation)
- **output**: JSON result data (returned by POST operation)

Operations **requiring JSON Tokens to be executed** in the context of authenticated sessions:

- **operation**: operation identifier (<opid>)
- **input**: JSON input data (arguments for the POST operation) and JSON-Tokens obtained for authenticated sessions
- **output**: JSON result data (returned by POST operation)

Obs) JSON tokens provided as input in authenticated requests must have been returned by the server component after a successful authentication, as result of a previous successful LOGIN REST POST operation.

For your exercise you will develop 10 (Ten) REST POST Operations supported in you cloud-enabled server-side component (with endpoints defined form the root of your endpoints), as defined in the next table:

Op. ID	Rest operation	Endpoint	Description	Requires input token ?
Op1	CreateAccount	/createaccount	Creates a user account (as registered account), given input attributes	No
Op2	Login	/login	Authenticates a user with a registered account and creates na authenticated session	No
OP3	ShowUsers	/showusers	Shows all users in registered accounts	Yes
Op4	DeleteAccount	/deleteaccount	Delete account	Yes
Op5	ModifyAccountAttributes	/modaccount	Modify one or more attributes in registered accounts	Yes
Op6	ShowAuthenticatedSessions	/showauthsessions	Show all authenticated sessions	Yes
Op7	ShowUserRole	/showuserrole	Show the role attribute of a user	Yes
Op8	ChangeUserRole	/changeuserrole	Changes the role of a user in its registration account	Yes
Op9	ChangeUserPassword	/changeuserpwd	Change a user password	Yes
Op10	Logout	/logout	Logout form an authenticated session	Yes

2.2 Rest Operations and Input/Output Specifications

General Rules

All operations use **HTTP POST**, with the following formats:

Request format:

```
{  
  "input": { ... },  
  "token": { ... }    // Only when required  
}
```

Response format on success

```
{  
  "status": "success",  
  "data": { ... }  
}
```

Response format on error:

```
{  
  "status": "<error>",      // <error> is the error code  
  "data": "<string>"        // the <string> is the error-message (corresponding to the code)  
}
```

JSON Token Structure

```
{  
  "tokenId": "string", // Generated randomly when token is created  
  "userId": "string",  
  "role": "ADMIN | BOFFICER | USER",  
  "issuedAt": "timestamp",  
  "expiresAt": "timestamp"  
}
```

Note that when a token is generated by the server-side (on a successful login operation), the token returned in the login operation must be also stored in a database (server-side) where you must manage persistently all issued tokens corresponding to authenticated user sessions. Tokens will be removed from this database in the logout operation.

Also note that all created accounts must be stored and managed in a database (server-side) where you will persistently maintain its relevant attributes.

2.3 Rest Operations and Error Coding

Descriptions and Common Error Codes

Error	Error Code	Error Message
INVALID_CREDENTIALS	9900	The username-password pair is not valid
USER_ALREADY_EXISTS	9901	Error in creating an account because the username already exists
USER_NOT_FOUND	9902	The username referred in the operation doesn't exist in registered accounts
INVALID_TOKEN	9903	The operation is called with an invalid token (wrong format for example)
TOKEN_EXPIRED	9904	The operation is called with a token that is expired
UNAUTHORIZED	9905	The operation is not allowed for the user role
INVALID_INPUT	9906	The call is using input data not following the correct specification
FORBIDDEN	9907	The operation generated a forbidden error by other reason

2.4 Rest Operations: Returned Error Codes for Different operations

The next table is a reference guide to follow, to manage possible errors that can be returned in the context of different operations

Op.Id	Endpoint	Input Json	Output Json	Errors (depending on the situation)
Op1	CreateAccount	<pre>{ "input": { "username": "string", "password": "string", "confirmation": "string", "email": "string", "phone": string, "address": string "role": "USER BOFFICER ADMIN" } }</pre> Notes: username: an unique identifier as a nickname (ex., "johnfrancis")	<pre>{ "status": "success", "data": { "username": "string", "role": "USER ADMIN" } }</pre>	USER_ALREADY_EXISTS INVALID_INPUT
Op2	Login	<pre>{ "input": { "username": "string", "password": "string" } }</pre>	<pre>{ "status": "success", "data": { "token": { "tokenId": "string", "userId": "string", "role": "USER ADMIN", "issuedAt": "timestamp", "expiresAt": "timestamp" } } }</pre>	INVALID_CREDENTIALS USER_NOT_FOUND INVALID_INPUT

Op3	ShowUsers	<pre>{ "input": {}, "token": { ... } }</pre>	<pre>{ "status": "success", "data": { "users": [{ "userId": "string", "username": "string", "email": "string", "role": "USER ADMIN" }] } }</pre>	INVALID_TOKEN INVALID_TOKEN TOKEN_EXPIRED UNAUTHORIZED FORBIDDEN
Op4	DeleteAccount	<pre>{ "input": { "userId": "string" }, "token": { ... } }</pre>	<pre>{ "status": "success", "data": { "message": "Account deleted successfully" } }</pre>	USER_NOT_FOUND INVALID_TOKEN TOKEN_EXPIRED UNAUTHORIZED FORBIDDEN

Op5	ModifyAccountAttributes	<pre>{ "input": { "userId": "string", "attributes": { "email": "string", "username": "string" } }, "token": { ... } }</pre>	<pre>{ "status": "success", "data": { "message": "Updated successfully" } }</pre>	USER_NOT_FOUND INVALID_TOKEN TOKEN_EXPIRED UNAUTHORIZED INVALID_INPUT FORBIDDEN
Op6	ShowAuthenticatedSessions	<pre>{ "input": {}, "token": { ... } }</pre>	<pre>{ "status": "success", "data": { "sessions": [{ "tokenId": "string", "userId": "string", "role": "string", "expiresAt": "timestamp" }] } }</pre>	INVALID_TOKEN INVALID_TOKEN TOKEN_EXPIRED UNAUTHORIZED FORBIDDEN
Op7	ShowUserRole	<pre>{ "input": { "userId": "string" }, "token": { ... } }</pre>	<pre>{ "status": "success", "data": { "userId": "string", "role": "USER ADMIN" } }</pre>	USER_NOT_FOUND INVALID_TOKEN TOKEN_EXPIRED UNAUTHORIZED FORBIDDEN

Op8	ChangeUserRole	<pre>{ "input": { "userId": "string", "newRole": "USER ADMIN" }, "token": { ... } }</pre>	<pre>{ "status": "success", "data": { "message": "Role updated successfully" } }</pre>	USER_NOT_FOUND INVALID_TOKEN TOKEN_EXPIRED INVALID_CREDENTIALS INVALID_TOKEN TOKEN_EXPIRED UNAUTHORIZED FORBIDDEN
Op9	ChangeUserPassword	<pre>{ "input": { "userId": "string", "oldPassword": "string", "newPassword": "string" }, "token": { ... } }</pre>	<pre>{ "status": "success", "data": { "message": "Password changed successfully" } }</pre>	INVALID_CREDENTIALS INVALID_TOKEN TOKEN_EXPIRED UNAUTHORIZED FORBIDDEN
Op10	Logout	<pre>{ "input": { "userID": "<username>", "token": { ... } } }</pre>	<pre>{ "status": "success", "data": { "message": "Logout successful" } }</pre>	INVALID_TOKEN TOKEN_EXPIRED UNAUTHORIZED FORBIDDEN

2.5 Rest Operations and Role-Based Access Control Policy

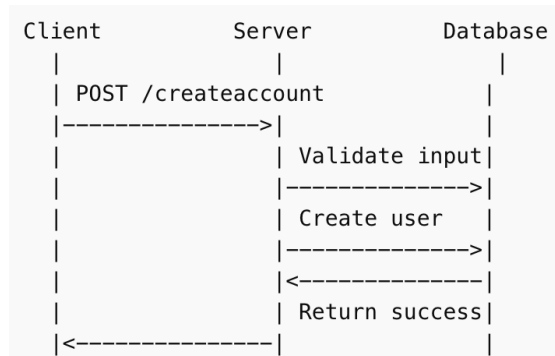
Operations are permitted or not allowed, depending on the role of the user that is calling the operation (with the respective Token previously obtained in a successful login). If an operation is not allowed for the user and his/her role, the UNAUTHORIZED error code must be returned.

Table with Reference for Access Control

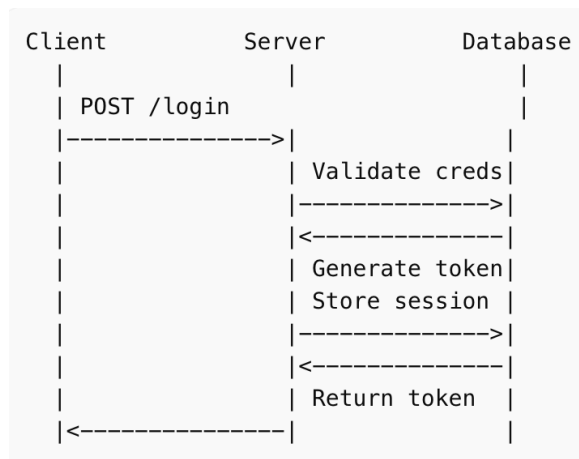
Op.Id	Endpoint	AUTHORIZED ROLES
Op1	CreateAccount	USER BOFFICER ADMIN
Op2	Login	USER BOFFICER ADMIN
Op3	ShowUsers	ADMIN BOFFICER
Op4	DeleteAccount	ADMIN
Op5	ModifyAccountAttributes	Username attribute cannot be changed (as it will be used as a primary key) USER (only can modify attributes in his/her own Account) BOFFICER (can modify attributes of his/her owned accounts or any USER accounts) ADMIN (can modify attributes of any account of any ROLE)
Op6	ShowAuthenticatedSessions	ADMIN
Op7	ShowUserRole	BOFFICER ADMIN
Op8	ChangeUserRole	ADMIN
Op9	ChangeUserPassword	USER BOFFICER ADMIN
Op10	Logout	USER (only logout of his/her sessions) BOFFICE (only logout of his/her sessions) ADMIN (logout for any session)

2.6 Reference Sequence Diagrams and Execution Flows for the Required Operations

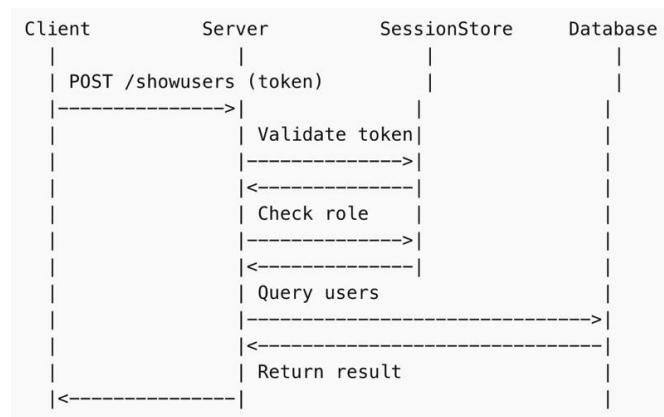
Op1. Create Account



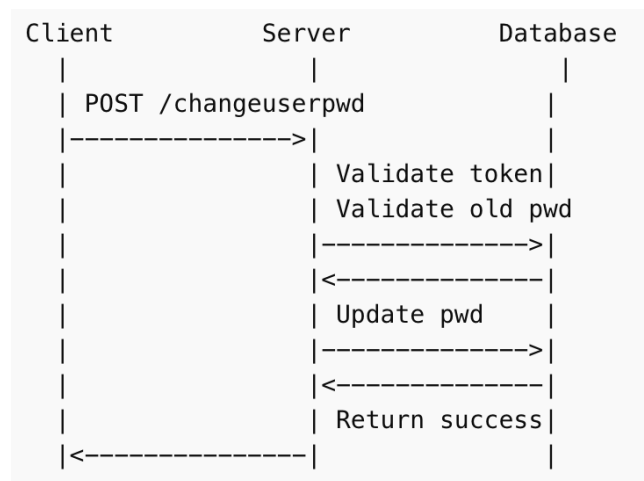
Op2. Login + Token Creation



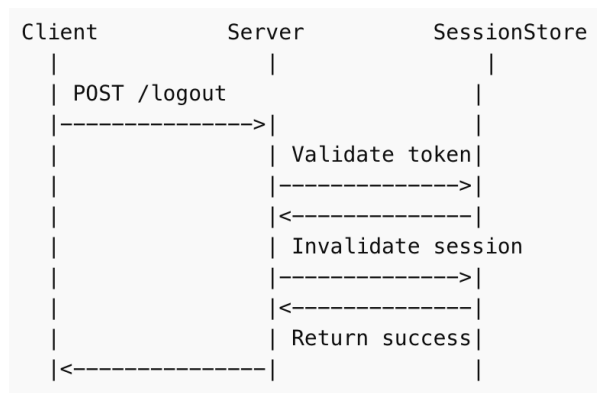
Op3 to Op8 : Authenticated Operation Flow (Generic Flow, here exemplified with `/showusers`)



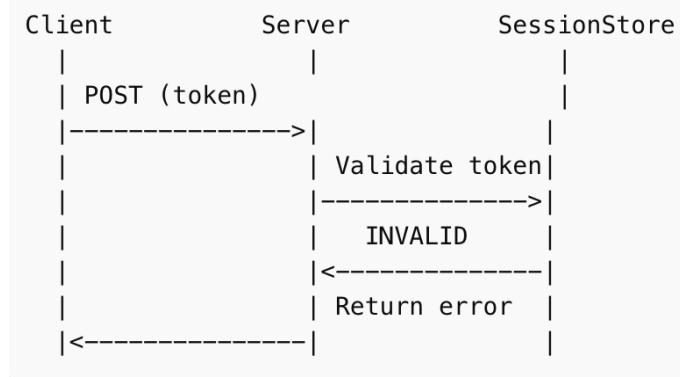
Op9 ChangePassword



Op10 Logout



Operation flow for token validation in all operations requiring the verification of exhibited tokens and returns in case of token errors



ANNEX: Example of test scripts to test POST endpoints

To test the REST operations in your implementation, you can use tools like Postman or Curl.

However (optionally), you can also automate your tests by calling the operations not necessarily “one-by-one” but as batch of calls (which can be faster and test sets of calls in the same batch). You are also free to think on a strategy for your tests. See also the next examples on how you can create scripts for tests.

Case: Simple examples using cURL in Windows (Command-Line, Terminal)

- **Access:** Open Command Prompt or PowerShell and type `curl --version` to ensure it is installed.
- **Basic GET Request:** Type `curl https://www.example.com` to fetch the HTML content of a website.
- **API POST Request:** Use `curl -X POST -d "param1=value1" https://api.example.com` to send data.

Case: using PowerShell and test scripts in Windows

```
# endpoints.ps1

# Define endpoints and payloads
$tests = @(
    @{
        Name = "Create User"
        Url = "https://api.example.com/users"
        Body = @{
            name = "John"
            email = "john@example.com"
        }
    },
    @{
        Name = "Login"
        Url = "https://api.example.com/login"
        Body = @{
            username = "john"
            password = "123456"
        }
    }
)

# Headers (optional)
$headers = @{
    "Content-Type" = "application/json"
```

```

    "Authorization" = "Bearer YOUR_TOKEN_HERE"
}
# Loop through tests
foreach ($test in $tests) {
    Write-Host "Running test: $($test.Name)" -ForegroundColor Cyan

    try {
        $jsonBody = $test.Body | ConvertTo-Json -Depth 10

        $response = Invoke-RestMethod `
            -Uri $test.Url `
            -Method Post `
            -Headers $headers `
            -Body $jsonBody

        Write-Host "Response:" -ForegroundColor Green
        $response | ConvertTo-Json -Depth 10

    } catch {
        Write-Host "Error:" -ForegroundColor Red
        Write-Host $_
    }
    Write-Host "-----"
}

```

How to Run

1. Save as endpoints.ps1
2. Open PowerShell
3. Run in your prompt (as follows)

```

// redirect the output overwriting the file
> .\endpoints.ps1 > results.txt
// redirect the output and appends to the file
> .\endpoibts.ps1 >> results.txt

```

```
// redirect the output and append to the file, not only the results but also errors  
> .\endpoints.ps1 >> output.txt
```

If scripts are blocked in your Windows OS and PowerShell environment you can unblock with:

```
> Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Case: using CuRL and shell test script in Linux (similar in MacOS)

```
#!/bin/bash

# File: test_endpoints.sh

# Base URL (optional)
BASE_URL="http://localhost:8080/api"

# Function to execute POST requests
call_endpoint() {
    NAME=$1
    URL=$2
    DATA=$3

    echo "====="
    echo "Running test: $NAME"
    echo "POST $URL"
    echo "Payload: $DATA"

    RESPONSE=$(curl -s -X POST "$URL" \
        -H "Content-Type: application/json" \
        -d "$DATA")

    echo "Response:"
    echo "$RESPONSE"
    echo ""
}

# ---- TEST CASES ----
```

```
call_endpoint "Create User" \  
"$BASE_URL/users" \  
'{"name":"John","email":"john@example.com"}'  
  
call_endpoint "Login" \  
"$BASE_URL/login" \  
'{"username":"john","password":"123456"}'  
  
call_endpoint "Create Order" \  
"$BASE_URL/orders" \  
'{"userId":1,"product":"Book","quantity":2}'  
  
echo "All tests completed."
```

How to have permissions for execution of test_endpoints.sh

```
$ chmod +x test_endpoints.sh
```

To run:

```
$ test_endpoints.sh
```